

```

1 100, last week | I author (100)
2 //! functions (procedural programming)
3 /*
4     ! notes :
5     |
6     | the function is responsible for performing a specific one task in a program
7
8     ~ it is a driven data type that is used to create objects (instances) of a class
9     ~ it is a block of code which only runs when it is called
10
11     You, 2 weeks ago • c++ , c
12
13     ~ the type of programming :
14     |
15     | # 1) functional programming (procedural programming)
16     | # 2) object-oriented programming (OOP)
17
18     ! function vs procedure :
19     |
20     | ~ a function is a block of code that performs a specific task and returns a value
21     |     # dont use print statements in a function that returns a value
22     |     $ dont read input from the user in a function that returns a value
23     |     $ send the data to the function as parameters instead
24
25     ~ a procedure is a block of code that performs a specific task but does not return a value
26     |     # use print statements in a procedure that does not return a value
27
28

```

```

21 ^ functional programming (procedural programming) :
22 ~ is a block of code which only runs when it is called
23 ! you can pass data , (known as parameters) into a function
24 ~ functions are used to perform certain actions ,
25 | (and they are also known as procedures , subroutines , methods , or routines to use a function , you need to )
26 ~ and they are important for reusing code :
27 | ! define the code once , and use it many times
28 # the code is organized in functions
29 # the data is separate from the code
30 # the code is reusable
31 # the code is easier to read and maintain
32
33 ? syntax of a function :
34 return_type function_name (parameters)
35 {
36     | // code to be executed
37 }
38 * return_type :
39 | ~ is the type of value that the function will return (e.g. int , double , string , etc.)\
40 | ~ when it implements it the code after the return statement will not be executed
41 | ! if the function does not return a value , you can use void as the return type
42 * function_name :
43 | ~ is the name of the function (e.g. add , subtract ,
44 * parameters :
45 | ~ is a list of variables that the function takes as input (e.g. int a , int b)
46 * code to be executed :
47 | ~ is the block of code that will be executed when the function is called
48

```

```

49  ? example of a function :
50      // procedural programming (dos not return a value)
51      void greet ()
52      {
53          cout << "Hello , World!" << endl;
54      }
55      // functional programming (return a value)
56      int add (int a , int b)
57      {
58          return a + b;
59      }
60      ! this function takes two integers as input and returns their sum as an integer
61      int main ()
62      {
63          int result = add (5 , 3);
64          cout << "The sum is : " << result << endl;
65          return 0;
66      }
67
68
69
70  */

```

```

1  //! variable scope
2  /*
3      ? the scope mean
4          % thr lifetime of a variable
5      ? the scope of a variable is the region of the program
6          ? where the variable is defined and can be accessed
7      ? there are three types of variable scope in C++ :
8          # 1) local scope
9              // the variable is defined inside a function or a block
10             // and can only be accessed within that function or block
11          # 2) global scope
12              // the variable is defined outside of all functions and can
13              // be accessed from any part of the program
14          * to avoid naming conflicts between local and global variables
15              %, you can use the scope resolution operator (::) to access the global variable
16          * to edit the global variable from a function ,
17              % you can use the scope resolution operator (::) to access the global variable
18          # 3) block scope
19              // the variable is defined inside a block (e.g. if statement , for loop , etc.)
20              // and can only be accessed within that block
21  */
22

```

```

23  ///! local scope
24  // example of local scope
25  #include <iostream>
26  using namespace std;
27  void myFunction()
28  {
29      int x = 10; // x is a local variable
30      cout << "The value of x is: " << x << endl;
31  }
    —— FUNCTION LINES: 2 ——
32
33  int main()
34  {
35      myFunction();
36      // cout << x; // error: x is not declared in this scope
37      return 0;
38  }
    —— FUNCTION LINES: 3 ——
39  ////////////////////////////////////
40

```

```

41  ///! global scope
42  // example of global scope
43  #include <iostream>
44  using namespace std;
45  int x = 10; // x is a global variable
46  void myFunction()
47  {
48      cout << "The value of x is: " << x << endl; // can access x
49  }
    —— FUNCTION LINES: 1 ——
50  int main()
51  {
52      myFunction();
53      cout << "The value of x is: " << x << endl; // can access x
54      return 0;
55  }
    —— FUNCTION LINES: 3 ——
56
57  // ! global scope with local variable
58  #include <iostream>
59  using namespace std;
60  int x = 10; // x is a global variable
61  void myFunction()
62  {
63      int x = 20; // x is a local variable that shadows the global variable
64      cout << "The value of x in myFunction is: " << x << endl; // will print 20
65  }
    —— FUNCTION LINES: 3 ——

```

```

66 int main()
67 {
68     int x = 30; // x is a local variable that shadows the global variable
69     myFunction();
70     cout << "The value of x in main is: " << x << endl; // will print 30
71
72     //~ access the global variable using the scope resolution operator
73     cout << "The value of global x is: " << ::x << endl; // can access the global variable using the scope resolution operator
74     //~ edit the global variable using the scope resolution operator
75     ::x = 40; // can edit the global variable using the scope resolution operator
76     cout << "The value of global x after editing is: " << ::x << endl; // can access the global variable using the scope resolution operator
77
78
79     return 0;
80 }
81

```

```

1  //! functions parameters - by value vs by reference
2  /*
3      ? when you pass an argument to a function,
4      ? you can do it in two ways:
5          1) by value
6          or
7          2) by reference.
8
9      # by value:
10         ? when you pass an argument by value, a copy of
11         ? the argument is created and passed to the function.
12         ? this means that any changes made to the parameter
13         ? inside the function do not affect the original
14         ? argument outside the function.
15
16     # by reference:
17         ? when you pass an argument by reference, the function
18         ? receives a reference to the original argument.
19         ? this means that any changes made to the parameter
20         ? inside the function affect the original argument
21         ? outside the function.
22         ! to pass an argument by reference,
23         ^ you can use the '&' symbol in the function parameter list.
24

```

```

25     ! note :
26     ~ if you use "&" simple inside cout it
27     ~ will print the address of the variable not its value,
28     ~ to print the value of the variable you should use the variable name without "&".
29 */
30
31 //~ example of passing by value
32 #include <iostream>
33 using namespace std;
34 int addByValue(int a, int b) {
35     return a + b;
36 }
37 int main() {
38     int x = 5, y = 10;
39     int result = addByValue(x, y);
40     cout << "Result of addByValue: " << result << endl; // Output: 15
41     cout << "Original x: " << x << ", Original y: " << y << endl; // Output: 5, 10
42     return 0;
43 }
44
45 //~ example of passing by reference
46 #include <iostream>
47 using namespace std;
48 void addByReference(int &a, int &b) {
49     a += b; // This will modify the original variable 'a'
50 }

```

```

51 int main() { 🐼 You, last week * ....
52     int x = 5, y = 10;
53     addByReference(x, y);
54     cout << "Result of addByReference: " << x << endl; // Output: 15
55     cout << "Original y: " << y << endl; // Output: 10
56     return 0;
57 }

```

```

C++ > C++ 16- Structures and Functions - Re-usability.cpp > ...
1  //! Structures and Functions - Re-usability
2  /*
3      // Re-usability :
4      ? means that we can use the same code again and again without writing it again and again.
5      ? it is one of the most important concepts in programming.
6
7      ~ note :
8      Re-usability is achieved through the use of functions and structures in C++.
9      Functions allow us to encapsulate a block of code that performs
10     a specific task, and we can call that function whenever we need to perform that task.
11     Structures allow us to group related data together, and we can create multiple
12     instances of that structure to store different sets of data.
13
14     ! your procedural functions
15     should be designed to perform a single task and return a value if needed,
16     so that they can be easily reused in different parts of your program or in
17     different programs altogether.
18
19
20 */

```

```

21
22 | 🐛
23 /// example of re-usability in C++ :
24 /*
25     ~ in this example we have a structure called Rectangle that has two members: length and width.
26     ~ we have a function called area that takes a Rectangle as a parameter
27     ~ and returns the area of the rectangle.
28     ~ we have another function called enterDimensions that takes a reference to a Rectangle
29     ~ and allows the user to enter the dimensions of the rectangle.
30     ~ we have another function called printArea that takes a Rectangle and prints
31     ~ the area of the rectangle.
32     ~ in the main function, we create a Rectangle object, call the
33     ~ enterDimensions function to fill it with data,
34     ~ and then call the printArea function to display the area of the rectangle.
35     ~ this demonstrates re-usability because we can use the area function and the printArea function
36     ~ with any Rectangle object without having to rewrite the code for calculating or
37     ~ printing the area each time.
38 */
39

```

```

40 #include <iostream>
41 using namespace std;
42
43 struct Rectangle
44 {
45     double length; 🐛
46     double width;
47 };
48
49 // procedural function to calculate the area of a rectangle
50 double area(Rectangle rect)
51 {
52     return rect.length * rect.width;
53 }
54 —— FUNCTION LINES: 1 ——
55
56 void enterDimensions(Rectangle &rect)
57 {
58     cout << "Enter the length and width of the rectangle: ";
59     cin >> rect.length >> rect.width;
60 }
61 —— FUNCTION LINES: 2 ——
62
63 void printArea(Rectangle rect)
64 {
65     cout << "The area of the rectangle is: "
66     << rect.length * rect.width << endl;
67 }
68 —— FUNCTION LINES: 2 ——

```

FUNCTION FILES 12

```
66
67 int main()
68 {
69     Rectangle rect;
70     enterDimensions(rect);
71     printArea(rect);
72
73     Rectangle rect2;
74     enterDimensions(rect2);
75     printArea(rect2);
76
77
78     return 0;
79 }
```